Day 1: What is Django & How It Works

Why do we even need Django?

Problem with only HTML/CSS/JS

If I ask:

- Save user data
- Login system
- Store notes
- Admin panel

Because:
Browser **cannot talk directly to database**

So we need:

```
Browser → Backend → Database
```

Django can:

- Receive request from browser
- Decide **what logic to run**
- Talk to database
- Send response back (HTML / JSON)

Django	Frontend
Backend framework	UI
Runs on server	Runs in browser
Uses Python	Uses HTML/CSS/JS

Django **generates HTML**, but it is still backend.

Django Architecture (VERY IMPORTANT)

Django follows **MVT**

Let’s rename it for understanding:

Model → Data (Database)

View → Brain (Logic)

Template→ Design (HTML)

Real-life analogy:

Amazon order

Part	Django	Real life
Data	**Model**	**Warehouse**
Logic	**View**	**Manager**
UI	**Template**	**Delivery box**

Django Project vs Django App (CONFUSION POINT)

Django Project

- Whole website
- Settings, configs

Django App

- One feature/module

Example:

Instagram (Project)

```
│
├── accounts (App)
├── posts (App)
├── chat (App)
```

One project = many apps

Starting with django:

1. Create virtual environment, activate it : **python3 -m venv virtualenvname / virtualenv name**
   Activate : for windows : name\Scripts\activate
   For mac: source name/bin/activate
2. **pip3 install django**
3. Run this command and check if it is installed : **django-admin --version**
4. **django-admin startproject project_name**
5. Use this to run the django file : **python manage.py runserver**

Folder structure you are seeing:

```
myproject/
│
├── manage.py
└── myproject/
```

Outer folder = project container
Inner folder = Django configuration

Django separates:

- Management commands (outer)
- Project configuration (inner)

So:

- `manage.py` stays outside
- Settings stay inside

manage.py

Purpose:

> This file is used to **run Django commands**.

Examples:

python manage.py runserver

python manage.py startapp appname

Analogy:

> Like npm for Django.

What is Admin Panel in Django?

Django Admin Panel is a ready-made dashboard to manage website data without writing extra code.

What can Admin Panel do?

From admin panel, you can:

- Add data
- Edit data
- Delete data
- Manage users
- Control permissions

All via **browser UI**, not code.

> Admin panel is **NOT for normal users**
> It is **ONLY for the website owner / developer**

Understanding `urls.py` (VERY IMPORTANT FILE)

from django.contrib import admin

Meaning:

Django provides an admin panel

This line imports it

If removed → admin won't work.

from django.urls import path

Meaning:

path() is used to define URLs

Without this, Django cannot map URLs

urlpatterns = []

Meaning:

A list of URL rules

Django checks this list top to bottom

Say:

“If URL is not here, Django doesn’t know it.”

path('admin/', admin.site.urls)

Meaning:

If URL starts with admin/

Send request to Django admin

User opens:

`http://127.0.0.1:8000/admin/`

Django:

- Matches `admin/`
- Sends request to `admin.site.urls`
- Admin panel opens

Adding Our Own Page

STEP 1: Import HttpResponse

Add:

```
from django.http import HttpResponse
```

Explanation:

- HTTP = how web works
- Response = what server sends back
- This class helps Django send text to browser

STEP 2: Create a View Function

Add:

```
def home(request):
    return HttpResponse("Hello Django")
```

Django automatically passes request

STEP 3: Connect URL to Function

Add:

```
path('', home)
```

Meaning:

- `''` → root URL
- `home` → function to call

Final `urls.py` (complete):

```
from django.contrib import admin
from django.urls import path
from django.http import HttpResponse


def home(request):
    return HttpResponse("Hello Django")
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', home),
]
```

Also try:

urls.py

```
from django.contrib import admin
from django.urls import path
from django.http import HttpResponse


def home(request):
   return HttpResponse("hello world")
def second_page(request):
   return HttpResponse("<h1>Second Page</h1>")
def third(request):
   return HttpResponse("third")
def second2(request):
   return HttpResponse("second 2")
urlpatterns = [
   path("admin/", admin.site.urls),
   path("",home),
   path("second/",second_page),
```

```
    path("third/",third),

    path("second/second_next",second2)

]
```

Why `views.py` Exists & What a Django App Is

“If our website has 50 pages…

will we write 50 functions inside `urls.py`?”

This is why Django **separates responsibilities**.

> urls.py decides WHERE to go
> views.py decides WHAT to do

What is a Django App?

Simple definition :

> **A Django app is a module that handles ONE feature of a website.**

Examples:

- blog
- notes
- accounts
- courses

One project → many apps.

Create Your First Django App

Run this command **from the folder where manage.py exists**:

python manage.py startapp main

Explain:

- startapp → create feature
- main → name of feature

Move logic OUT of urls.py

views.py ->

```
from django.shortcuts import render

from django.http import HttpResponse


# Create your views here.

def home(request):

    return HttpResponse("hey")

def second_page(request):

    return HttpResponse("second page")
```

urls.py->

```
from django.contrib import admin

from django.urls import path
```

```
from main import views


urlpatterns = [
    path("admin/", admin.site.urls),
    path("",views.home),
    path("second/",views.second_page)
]
```

FINAL MENTAL MODEL

Browser

↓

urls.py → decides route

↓

views.py → runs logic

↓

HttpResponse

↓

Browser

When using apps, Django expects templates inside the app folder.

When we create an app using:

```
python manage.py startapp main
```

Django **does NOT automatically use it**.

In settings.py:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    'main',
]
```

We must register it.

Correct folder structure:

main/

├── views.py

├── templates/

│ └── home.html

```
<!DOCTYPE html>

<html>

<head>

   <title>Home</title>

</head>

<body>

   <h1>Welcome to Django</h1>

   <p>This page is coming from a template.</p>

</body>

</html>
```

views.py

```
from django.shortcuts import render

from django.http import HttpResponse



# Create your views here.

def home(request):
```

```
    return render(request,"home.html")

def second_page(request):

    return HttpResponse("second page")
```

Browser

↓

urls.py → decides route

↓

views.py → calls render()

↓

templates/home.html

↓

Browser shows HTML

DAY 2 – CHAPTER 2

Passing Data from Views → Templates (DTL Basics)

Remember:

Data Flow Diagram

views.py
↓ (context dictionary)
template.html
↓
browser

> "Views send data, templates receive data."

Passing ONE Variable

views.py

```
def home(request):

    name="bhavna"

    return render(request,"home.html",{"n":name})
```

home.html

```
<h1>Welcome to Django {{n}}</h1>
```

views.py

```
def home(request):

    name="bhavna"

    x=8
```

```
    y=9

    z=0

    return
render(request,"home.html",{"n":name,"x":x,"y":y,"z":z})
```

Home.html

```
<h1>Welcome to Django {{n}} {{x}} {{y}} {{z}}</h1>
```

views.py

```
def home(request):
    context = {
        "name": "Ram",
        "course": "Django",
        "city": "Delhi"
    }
    return render(request, "home.html", context)
```

home.html

```
<p>Name: {{ name }}</p>
<p>Course: {{ course }}</p>
<p>City: {{ city }}</p>
```

Passing LIST Data

```
def home(request):
    subjects = ["HTML", "CSS", "JS", "Django"]
    return render(request, "home.html", {"subjects": subjects})
```

Using {% for %} Loop in Template

home.html

```
<h3>Subjects</h3>
<ul>
   {% for sub in subjects %}
       <li>{{ sub }}</li>
   {% endfor %}
</ul>
```

Using {% if %} Condition

views.py

```
def home(request):
   is_logged_in = True
   return render(request, "home.html", {"is_logged_in":
is_logged_in})
```

home.html

```
{% if is_logged_in %}
   <p>Welcome back!</p>
{% else %}
   <p>Please login</p>
{% endif %}
```

Template Inheritance

Each page has:

- Header
- Menu
- Footer

So HTML looks like this:

home.html → header + content + footer
about.html → header + content + footer
contact.html→ header + content + footer

Template inheritance means one common HTML file that other pages reuse.

That common file is called:

base.html

Folder Structure

templates/

├── base.html

├── home.html

└── about.html

Create `base.html`

```
<!DOCTYPE html>

<html>

<head>

   <title>My Website</title>

</head>
```

```
<body>

    <h2>This is Header</h2>

    {% block content %}
    {% endblock %}

    <h4>This is Footer</h4>

</body>
</html>
```

Create home.html (child template)

```
{% extends "base.html" %}

{% block content %}
    <h1>Home Page</h1>
    <p>This is home content</p>
{% endblock %}
```

Create about.html (same pattern)

```
{% extends "base.html" %}

{% block content %}
   <h1>About Page</h1>
   <p>This is about content</p>
{% endblock %}
```

Day 3: ORM and databases

First Understand the Problem

Right now, where is our data stored?

Answer:

- Inside **`views.py`**
- Hardcoded
- Temporary

Example:

```
name = "Ram"
```

If we stop server → data is gone
If 100 students use website → everyone sees same data

So we need:

A place to store data permanently.

That is database.

What is a Database?

A database is like:

- Excel sheet
- Or notebook table
- Or register

Example table:

id	name	course	age
1	Ram	Django	20
2	Sita	Python	22

This table stores data permanently.

What is a Model?

Now say this carefully:

> A Model is a Python class that tells Django
> how the database table should look.

Important:

- Model = Blueprint of table
- Not actual table yet
- Just design

Create First Model

Open:

`main/models.py`

You will see:

```
from django.db import models
```

Do not delete it.

Now add this below:

```
class Student(models.Model):
    name = models.CharField(max_length=100)
```

`class Student(models.Model):`

- Creating a class
- Name = Student
- Inheriting from Django's Model class

This tells Django:

> I want a table called Student.

`name = models.CharField(max_length=100)`

This means:

- Column name = name
- Type = text
- Max 100 characters

Simple translation:

> Student table will have a name column.

Did we create the table in database yet? No.

Why Database Does Not Update Automatically?

Because:

- Writing model is like drawing table design.

- Database needs instructions to create it.

Django gives two commands for that.

Run:

```
python manage.py makemigrations
```

Django checks models

Finds new model

Creates migration file

You will see something like:

```
Migrations for 'main':
  main/migrations/0001_initial.py
```

This file contains:

- SQL instructions
- Table creation logic

But table still not created.

Apply Migration

Run:

```
python manage.py migrate
```

- Django reads migration file
- Executes SQL
- Creates actual table in database

Now table exists.

Where Is Database?

Look in project folder:

You will see:

```
db.sqlite3
```

This is your database file.

Say:

> SQLite is default database for Django.

No configuration needed for now.

models.py → makemigrations → migrate → database table created

Register Model in Admin (Small Step)

Open:

```
main/admin.py
```

Add:

```
from .models import Student


admin.site.register(Student)
```

- We are telling admin panel
- Show Student table

Create Superuser (Only Once)

Run:

```
python manage.py createsuperuser
```

A superuser is an admin account that has **full control** over the website.

Follow prompts.

Then go to:

```
http://127.0.0.1:8000/admin/
```

Login.

You should see:
Student

Click it.

Add a student.

Save.

You built a website.

There are:

- Normal users (students, customers)
- Admin users (people who manage the site)

Superuser = **highest level admin**

Superuser allows you to:

- Add data

- Edit data
- Delete data
- Manage users
- Control permissions

Now Let’s Add Data Properly

Step 1: Open Admin Panel

Go to:

```
http://127.0.0.1:8000/admin/
```

Login with superuser.

Step 2: Click “Students”

You should see:

```
Students
```

Click it.

Step 3: Click “Add Student”

Fill form:

- Name: Ram

- Age: 20

Click Save.

Step 4: Add 2–3 Students

Add:

- Ram
- Sita
- Mohan

Step 5: Refresh Your Website

Open:

`http://127.0.0.1:8000/admin`

Now you should see list of students.

Now we will:

> **Read those students from database**
> **and show them on the website.**

Understand the Flow

```
Database

    ↓
```

```
Model
    ↓
View
    ↓
Template
    ↓
Browser
```

Import Model in `views.py`

Open:

`main/views.py`

At the top, add:

```
from .models import Student
```

- `.` means current app
- **`models`** means models.py
- **`Student`** is the model class

Fetch Data from Database

Inside your **`home`** view, write:

```python
from .models import Student

def home(request):

    students = Student.objects.all()

    return render(request, "home.html", {"students": students})
```

Update Template to Display Students

Open:

main/templates/home.html

Add->

```html
<h2>Student List</h2>



<ul>

    {% for student in students %}

        <li>{{ student.name }}</li>

    {% endfor %}

</ul>
```

What Are Field Types?

Field types decide:

What kind of data can be stored in a column.

Think Excel:

- Text column
- Number column
- Date column
- Boolean column

Django fields are similar.

CharField

TextField

IntegerField

EmailField

BooleanField

DateField

DateTimeField

CharField

```
name = models.CharField(max_length=100)
```

Used for:

- Name
- Title
- Course
- City

Important:

- **`max_length`** is required

Think:

> Short text.

TextField

```
description = models.TextField()
```

Used for:

- Long paragraphs
- Blog content
- Notes

Difference from CharField:

CharField	TextField
Short text	Long text
max_length required	No max_length

IntegerField

```
age = models.IntegerField()
```

Used for:

- Age
- Marks
- Quantity

Stores only whole numbers.

EmailField

```
email = models.EmailField()
```

Used for:

- Email addresses

Benefit:

- Validates proper email format
- Admin form checks email structure

Better than CharField for emails.

BooleanField

```
is_active = models.BooleanField(default=True)
```

Used for:

- True / False
- Yes / No
- Published / Not Published

`default=True`:
If no value provided → True automatically

DateField

```
date_of_birth = models.DateField()
```

Stores:

- Only date
- No time

Format:
YYYY-MM-DD

DateTimeField

```
created_at = models.DateTimeField(auto_now_add=True)
```

Stores:

- Date + Time

Important options:

`auto_now_add=True`

Automatically stores current time when object is created.

`auto_now=True`

Updates time every time object is saved.

Important Field Options

null=True

Database allows empty value.

blank=True

Form allows empty input.

Difference:

- `null` → database level
- `blank` → form level

default=

```
age = models.IntegerField(default=18)
```

If no value provided → 18 automatically.

unique=True

```
email = models.EmailField(unique=True)
```

No two rows can have same email.

Database enforces uniqueness.

Simple Example Model

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    age = models.IntegerField()
    bio = models.TextField(blank=True)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
```

Task: Student Management Portal (Basic Version)

Project Goal

Build a simple website that:

- Stores students in database
- Shows student list on homepage
- Shows student details
- Uses template inheritance
- Uses real data from database

Filtering Data (Using Django ORM)

Step 0: Why Do We Need Filtering?

Right now, we are using:

```
students = Student.objects.all()
```

That means:

> Give me ALL students.

But real apps need:

- Only Django course students
- Only active students
- Only students age > 20
- Only specific student

So we need:

Filtering.

Step 1: Basic Filtering with `.filter()`

Open **`views.py`**.

Example:

```
students = Student.objects.filter(course="Django")
```

Explain slowly:

- **`Student`** → table
- **`objects`** → manager
- **`.filter()`** → apply condition

This means:

SELECT * FROM student WHERE course = 'Django';

But we didn't write SQL.

ORM did.

Important Concept

`.filter()` always returns:

QuerySet (like a list)

Even if:

- 0 results
- 1 result
- 10 results

It still returns a QuerySet.

That's important.

Step 2: Show Filtered Data in Template

In **`views.py`**:

```
def home(request):

    students = Student.objects.filter(course="Django")
```

```
    return render(request, "home.html", {"students": students})
```

No template change needed.

It works same way.

Step 3: Filtering by Number

Example:

```
students = Student.objects.filter(age=20)
```

SQL equivalent:

```
WHERE age = 20
```

Step 4: Filtering by Boolean

```
students = Student.objects.filter(is_active=True)
```

Very common in real apps.

Step 5: Greater Than / Less Than (Important)

Django uses special syntax.

Greater than:

```
students = Student.objects.filter(age__gt=18)
```

Less than:

```
students = Student.objects.filter(age__lt=25)
```

Break this:

- **`age`** → field
- **`__gt`** → greater than

Common Lookups

Lookup	Meaning
`__gt`	greater than
`__lt`	less than
`__gte`	greater than equal
`__lte`	less than equal
`__contains`	text contains
`__icontains`	case-insensitive contains

Example: Search by Name

```
students = Student.objects.filter(name__icontains="ra")
```

This finds:

- Ram
- Ramesh
- Ratan

Case-insensitive.

Very Important Difference

`.filter()`

Returns many objects.

`.get()`

Returns exactly ONE object.

If no object → error
If multiple objects → error

So:

Use **`.get()`** only when expecting one result.

Step 6: Multiple Conditions

Example:

```
students = Student.objects.filter(course="Django", age__gt=18)
```

Means:

course = Django AND age > 18

Step 7: Exclude Data

Opposite of filter:

```
students = Student.objects.exclude(course="Django")
```

Means:

> Give me all students except Django course.

Step 8: Ordering Data

```
students = Student.objects.order_by("age")
```

Ascending.

Descending:

```
students = Student.objects.order_by("-age")
```

Mental Model for QuerySets

Think of QuerySet like:

- Smart list
- Lazy database query
- Chainable

Example:

```
students =
Student.objects.filter(course="Django").order_by("-age")
```

You can chain them.

Chapter: Create, Update, Delete Using ORM

Right now:

We created data using admin
We read data using **`.all()`** and **`.filter()`**

Now we learn:

- Create using Python
- Update using Python
- Delete using Python

PART 1: Creating Objects Using ORM

Method 1 (Most Common)

Open **`views.py`** and write:

```
from .models import Student


def create_student(request):
    student = Student(
        name="Rahul",
        email="rahul@gmail.com",
        age=22,
        course="Django"
    )
    student.save()
```

```
    return HttpResponse("Student Created")
```

Step-by-step:

1. Create Python object
2. Call `.save()`
3. Django writes SQL
4. Record inserted into database

SQL equivalent:

```
INSERT INTO student ...
```

Method 2 (Shortcut)

```
Student.objects.create(
    name="Aman",
    email="aman@gmail.com",
    age=21,
    course="Python"
)
```

This does:

- Create object
- Save automatically

Add URL for Testing

In **`urls.py`**:

```
path("create/", views.create_student),
```

Visit:

```
/create/
```

Now student is created.

Important Concept

Data is now inserted without admin.

This is real backend logic.

PART 2: Updating Data

Suppose we want to update student's age.

Step 1: Get Object

```
student = Student.objects.get(id=1)
```

Step 2: Modify Field

```
student.age = 25
```

Step 3: Save Again

```
student.save()
```

Complete example:

```
def update_student(request):
    student = Student.objects.get(id=1)
    student.age = 30
    student.save()

    return HttpResponse("Student Updated")
```

PART 3: Deleting Data

Step 1: Get Object

```
student = Student.objects.get(id=1)
```

Step 2: Delete

```
student.delete()
```

Complete example:

```
def delete_student(request):
    student = Student.objects.get(id=1)
    student.delete()

    return HttpResponse("Student Deleted")
```

Full CRUD Summary

Operation	ORM Method
Create	`.create()` or `.save()`
Read	`.all()`, `.filter()`, `.get()`
Update	**Modify + `.save()`**
Delete	`.delete()`

Forms – Let Users Create Data from Browser

Until now:

- Admin created data
- ORM created data
- But user cannot create data from website

Now we fix that.

There are TWO types of HTTP requests:

Method	Purpose
GET	Request data

POST Send data

When submitting form: We use **POST**

Step 1: Create Add Student Page

1. Create URL

In **`urls.py`**:

path("add-student/", views.add_student),

2. Create View

In **`views.py`**:

```
from django.shortcuts import render, redirect
from .models import Student

def add_student(request):
    if request.method == "POST":
        name = request.POST.get("name")
        email = request.POST.get("email")
        age = request.POST.get("age")
        course = request.POST.get("course")

        Student.objects.create(
            name=name,
            email=email,
            age=age,
            course=course
        )

        return redirect("home")  # or redirect("/")

    return render(request, "add_student.html")
```

`request.method == "POST"`

Means:

Form was submitted.

`request.POST.get("name")`

Means:

Get value from input field.

`name` must match HTML input name.

`Student.objects.create()`

Creates record in database.

`redirect("home")`

After successful creation, go back to homepage.

Step 3: Create Template

Create:

add_student.html

Write:

{% extends "base.html" %}

{% block content %}

<h3>Add Student</h3>

```
<form method="POST">
   {% csrf_token %}

   <input type="text" name="name" placeholder="Name"><br><br>
   <input type="email" name="email" placeholder="Email"><br><br>
   <input type="number" name="age" placeholder="Age"><br><br>
   <input type="text" name="course" placeholder="Course"><br><br>

   <button type="submit">Add Student</button>
</form>

{% endblock %}
```

Very Important: CSRF Token

{% csrf_token %}

This prevents security attacks.

If you remove it:

You will get:

403 Forbidden (CSRF verification failed)

Never remove it.

Step 4: Test It

1. Run server

Open:
/add-student/

2.
3. Fill form
4. Submit
5. Check homepage

6. Check admin panel

Student should appear.

What Just Happened?

User fills form
↓
POST request
↓
Django receives data
↓
ORM creates record
↓
Redirect to homepage

Real full-stack flow.

Add simple validation:

```
if not name:
    return render(request, "add_student.html", {"error": "Name required"})
```

Then in template:

```
{% if error %}
<p style="color:red;">{{ error }}</p>
{% endif %}
```

ALso read :

https://docs.djangoproject.com/en/6.0/topics/forms/

Model Forms

1. First Understand the Problem

Earlier, when we created a form manually, we did something like this:

HTML:

```
<form method="POST">
    {% csrf_token %}
    <input type="text" name="name">
    <input type="email" name="email">
    <button type="submit">Save</button>
</form>
```

In views.py:

```
def add_student(request):
    if request.method == "POST":
        name = request.POST.get("name")
        email = request.POST.get("email")

        Student.objects.create(name=name, email=email)

        return redirect("home")

    return render(request, "add_student.html")
```

Problems in this approach:

1. We manually extract data.
2. We manually validate data.
3. We manually create object.
4. If model changes, we must update form.
5. Too much repeated code.

Django provides a better way.

That better way is ModelForm.

---

2. What is ModelForm?

Definition:

A ModelForm is a form that is automatically created from a model.

It connects:

Model → Form → Database

This means:

If we have a model, Django can automatically:

- Create form fields
- Validate data
- Save data into database

---

3. Prerequisite: You Must Have a Model

Example model:

models.py

```
from django.db import models


class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    age = models.IntegerField()
    course = models.CharField(max_length=100)


    def __str__(self):
        return self.name
```

Make sure you already ran:

```
python manage.py makemigrations
python manage.py migrate
```

`__str__` is a special Python method.

It defines how an object is displayed as a string.

In Django models, it controls how objects appear:

- In Django admin
- In Django shell
- In dropdowns (ForeignKey)
- When printed

Without `__str__`

Model:

```
class Student(models.Model):

    name = models.CharField(max_length=100)
```

If you open admin panel, you might see:

```
Student object (1)

Student object (2)
```

This is not helpful.

With `__str__`

```
class Student(models.Model):

    name = models.CharField(max_length=100)


    def __str__(self):

        return self.name
```

Now admin panel shows:

Ram
Aman
Sita

Much clearer.

Why is `self` used?

`self` refers to the current object.

If student name is "Ram":

`self.name → "Ram"`

---

4. Create forms.py File

Inside your app folder (for example: core/), create a new file:

`forms.py`

This file does not exist automatically. You create it manually.

---

5. Create Your First ModelForm

Inside forms.py:

```
from django import forms

from .models import Student


class StudentForm(forms.ModelForm):


    class Meta:

        model = Student

        fields = '__all__'
```

Now understand each line carefully.

---

6. Explanation of Each Part

Line 1:

```
from django import forms
```

We are importing Django's form system.

Line 2:

```
from .models import Student
```

We are importing our model.

Line 4:

```
class StudentForm(forms.ModelForm):
```

We are creating a new form class.

It is not a normal form.

It is a ModelForm.

Inside the class:

```
class Meta:
```

Meta is a configuration class.

It tells Django:

- Which model to use
- Which fields to include

This line:

```
model = Student
```

Means:

This form is based on Student model.

This line:

```
fields = '__all__'
```

Means:

Include all model fields in the form.

You can also write:

```
fields = ['name', 'email', 'age']
```

If you want only specific fields.

---

7. Using ModelForm in View

Now go to views.py.

Replace manual form code with this:

```
from django.shortcuts import render, redirect
```

```
from .forms import StudentForm


def add_student(request):


    if request.method == "POST":

        form = StudentForm(request.POST)


        if form.is_valid():

            form.save()

            return redirect("home")


    else:

        form = StudentForm()


    return render(request, "add_student.html", {"form": form})
```

Now understand this clearly.

---

8. Step-by-Step Explanation of View

Step 1:

```
form = StudentForm(request.POST)
```

We are binding form with submitted data.

Step 2:

```
if form.is_valid():
```

Django checks:

- Required fields
- Email format
- Integer type
- Unique constraints
- Max length

If something is wrong, form is not valid.

Step 3:

```
form.save()
```

This automatically:

- Creates Student object
- Saves it in database

We do not need:

Student.objects.create()

Step 4 (GET request):

```
form = StudentForm()
```

This creates an empty form for the user.

---

9. Update Template

Now change add_student.html:

```
<form method="POST">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Save</button>
</form>
```

Explanation:

- {{ form.as_p }} automatically generates all input fields.
- Django creates correct input types.
- It shows validation errors automatically.

---

10. What Happens If Data Is Invalid?

Example:

- Leave name empty
- Enter invalid email

Django will:

- Not save data
- Show error messages
- Keep previously entered values

You do not write extra code.

---

11. Difference Between Manual Form and ModelForm

Manual Form:

- Extract data manually
- Validate manually
- Save manually

ModelForm:

- Auto-generate fields
- Auto-validate
- Auto-save

Much cleaner and professional.

Excluding Fields

If your model has:

```
created_at = models.DateTimeField(auto_now_add=True)
```

You do not want user to fill this.

Use:

```
class Meta:
    model = Student
    exclude = ['created_at']
```

Instead of fields.

Access Cleaned Data (Advanced Concept)

After validation:

```
if form.is_valid():
    print(form.cleaned_data)
```

cleaned_data is a dictionary of validated data.

Example:

```
name = form.cleaned_data['name']
```

PART 2 - Update Using ModelForm

1. What is Update?

We already learned Create.

Now we want to:

- Load existing data
- Show it in form
- Modify it

- Save changes

---

2. URL for Update

In urls.py:

```
path("update/<int:id>/", views.update_student,
name="update_student"),
```

---

3. View for Update

```
from django.shortcuts import get_object_or_404

from .forms import StudentForm


def update_student(request, id):


    student = get_object_or_404(Student, id=id)


    if request.method == "POST":

        form = StudentForm(request.POST, instance=student)


        if form.is_valid():

            form.save()

            return redirect("home")
```

```
    else:
        form = StudentForm(instance=student)

    return render(request, "add_student.html", {"form": form})
```

---

4. Important Concept: **`instance=student`**

This tells Django:

"This form is editing this existing object."

Without instance, it creates a new object.

---

5. Template Remains Same

```
<form method="POST">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Update</button>
</form>
```

---

6. Flow

User clicks edit
→ Student loaded
→ Form pre-filled
→ User edits

→ POST
→ form.save() updates record

PART 3 - Custom Validation

1. Why Custom Validation?

Sometimes built-in validation is not enough.

Example:

- Age must be greater than 18
- Name cannot contain numbers

---

2. Field-Level Validation

Inside forms.py:

```
class StudentForm(forms.ModelForm):


    class Meta:

        model = Student

        fields = '__all__'


    def clean_age(self):

        age = self.cleaned_data.get('age')


        if age < 18:
```

```
            raise forms.ValidationError("Age must be at least 18.")


        return age
```

Explanation:

- `clean_age` validates only age field.
- Must return value if valid.
- Raise ValidationError if invalid.

---

3. Form-Level Validation

```
def clean(self):
    cleaned_data = super().clean()


    name = cleaned_data.get("name")
    course = cleaned_data.get("course")


    if name and course:
        if name.lower() == course.lower():
            raise forms.ValidationError("Name and course cannot be same.")


    return cleaned_data
```

This validates multiple fields together.

---

4. Important Rule

Always return cleaned_data.

PART 4 - Styling ModelForm Fields

By default:

```
{{ form.as_p }}
```

Generates plain HTML.

We can add CSS classes.

---

1. Using widgets

In forms.py:

```
class StudentForm(forms.ModelForm):


    class Meta:

        model = Student

        fields = '__all__'

        widgets = {

            'name': forms.TextInput(attrs={'class':
'form-control'}),
```

```
            'email': forms.EmailInput(attrs={'class':
'form-control'}),

            'age': forms.NumberInput(attrs={'class':
'form-control'}),

            'course': forms.TextInput(attrs={'class':
'form-control'}),

        }
```

---

2. What This Does

It adds class="form-control" to input fields.

Useful when using Bootstrap or Tailwind.

---

3. Manual Rendering (More Control)

Instead of:

```
{{ form.as_p }}
```

You can do:

```
{{ form.name.label_tag }}

{{ form.name }}

{{ form.name.errors }}
```

This gives full control.

PART 5 - Delete Confirmation Page

1. Why Confirmation?

Direct delete is risky.

Better approach:

Step 1: Show confirmation page
Step 2: User confirms
Step 3: POST request deletes

---

2. URL

```
path("delete/<int:id>/", views.delete_student,
name="delete_student"),
```

---

3. View

```
def delete_student(request, id):


    student = get_object_or_404(Student, id=id)


    if request.method == "POST":

        student.delete()

        return redirect("home")


    return render(request, "delete_confirm.html", {"student":
student})
```

4. delete_confirm.html

```
{% extends "base.html" %}


{% block content %}


<h3>Are you sure you want to delete {{ student.name }}?</h3>


<form method="POST">
    {% csrf_token %}
    <button type="submit">Yes, Delete</button>
    <a href="{% url 'home' %}">Cancel</a>
</form>


{% endblock %}
```

CHAPTER: Sending Email in Django

PART 1 — How Email Works in Django

Django does not send email automatically.

It uses:

- SMTP server (Gmail, Outlook, etc.)
- Email backend configuration
- send_mail() function

So there are 3 steps:

1. Configure email settings
2. Create form to take input
3. Use send_mail() in view

PART 2 — Configure Email in settings.py

Open settings.py and add this at the bottom:

EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'

EMAIL_HOST = 'smtp.gmail.com'

EMAIL_PORT = 587

EMAIL_USE_TLS = True

EMAIL_HOST_USER = 'your_email@gmail.com'

EMAIL_HOST_PASSWORD = 'your_app_password'

Important:

Do NOT use your real Gmail password.

You must create an App Password in Gmail.

Getting app password :

1. Open accounts.google.com
2. In security, turn on 2 step verification
3. Search for app password , type app name anything you want ,generate
4. Copy the password

PART 3 - Create Email Form (Using ModelForm? No.)

Email sending is not stored in database.

So we use a normal Form.

Create in forms.py:

```
from django import forms


class EmailForm(forms.Form):

    sender = forms.EmailField()

    receiver = forms.EmailField()

    subject = forms.CharField(max_length=200)

    message = forms.CharField(widget=forms.Textarea)
```

Explanation:

- forms.Form (not ModelForm)
- EmailField validates email format
- Textarea gives large input box

PART 4 - Create View to Send Email

In views.py:

```
from django.core.mail import send_mail

from django.conf import settings

from .forms import EmailForm


def send_email_view(request):


    if request.method == "POST":

        form = EmailForm(request.POST)


        if form.is_valid():
```

```
        sender = form.cleaned_data['sender']
        receiver = form.cleaned_data['receiver']
        subject = form.cleaned_data['subject']
        message = form.cleaned_data['message']

        send_mail(
            subject,
            message,
            sender,
            [receiver],
            fail_silently=False,
        )

        return redirect("dashboard")

    else:
        form = EmailForm()

    return render(request, "send_email.html", {"form": form})
```

Explanation of send_mail():

```
send_mail(
    subject,
    message,
```

```
    from_email,
    recipient_list,
)
```

Important:

recipient_list must be a list.

Even for one email:

[receiver]

PART 5 - Create Template

send_email.html

```
{% extends "base.html" %}

{% block content %}

<h3>Send Email</h3>

<form method="POST">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Send</button>
</form>

{% endblock %}
```

PART 6 - Add URL

In urls.py:

path("send-email/", views.send_email_view, name="send_email"),

Now test:

/send-email/

PART 7 - Very Important Security Note

Students should understand:

We are allowing user to enter sender email.

But in real applications:

You normally use:

settings.EMAIL_HOST_USER

as sender.

Users should not send from arbitrary emails.

Safer version:

```
send_mail(
    subject,
    message,
    settings.EMAIL_HOST_USER,
    [receiver],
```

)

Authentication

Lets start with manual authentication first

urls.py

```
from django.contrib import admin

from django.urls import path

from main import views



urlpatterns = [

    path("admin/", admin.site.urls),

    path("",views.home),

    path("register/",views.add_user,name="register"),

    path("login/",views.login_check,name="login"),

    path("logout/",views.logout_view,name="logout"),

    path("dashboard/",views.dashboard,name="dashboard"),

]
```

Home.html

```
<html>

    <head>

        <title>this is home page</title>

    </head>
```

```
    <body>
        <h1>hello</h1>
    </body>
</html>
```

Models.py

```
from django.db import models


# Create your models here.
class User(models.Model):
    username=models.CharField(max_length=100,unique=True)
    password=models.CharField(max_length=20)


    def __str__(self):
        return self.username
```

admin.py

```
from django.contrib import admin
from .models import User
# Register your models here.
admin.site.register(User)
```

Register.html

```
<html>
```

```
    <head>
        <title>this is home page</title>
    </head>
    <body>
        <h2>Register</h2>
        <form method="post">
            {% csrf_token %}
            <input type="text" name="username"
placeholder="Username">
            <input type="password" name="password"
placeholder="Password">
            <button type="submit">Register</button>
        </form>
    </body>
</html>
```

Login.html

```
<html>
    <head>
        <title>this is home page</title>
    </head>
    <body>
        <h2>Login</h2>
```

```
        <form method="post">
            {% csrf_token %}
            <input type="text" name="username"
placeholder="Username">
            <input type="password" name="password"
placeholder="Password">
            <button type="submit">Login</button>
        </form>
        {% if error %}
        <p>{{ error }}</p>
        {% endif %}
    </body>
</html>
```

Logout.html

```
<html>
    <head>
        <title>this is home page</title>
    </head>
    <body>
        <h2>Logout</h2>
        <form method="post">
            {% csrf_token %}
```

```
            <button type="submit">Logout</button>
        </form>
    </body>
</html>
```

Dashboard.html

```
<html>
    <head>
        <title>this is home page</title>
    </head>
    <body>
        <h2>hello {{ user.username }}</h2>
        </form>
    </body>
</html>
```

views.py

```
from django.shortcuts import render,redirect
from .models import User
# Create your views here.
def home(request):
    return render(request,"home.html")


def add_user(request):
```

```
    if request.method=="POST":
        username=request.POST.get("username")
        password=request.POST.get("password")



User.objects.create(username=username,password=password)
        return redirect("/")
    return render(request,"register.html")


def login_check(request):
    if request.method=="POST":
        username=request.POST.get("username")
        password=request.POST.get("password")
        try:
            user=User.objects.get(username=username)
            if user.password==password:
                request.session["user_id"] = user.id
                return redirect("dashboard")
            else:
                return
render(request,'login.html',{"error":"Invalid password"})
        except User.DoesNotExist:
```

```
            return render(request,"login.html",{"error":"User
does not exist"})

    return render(request,"login.html")


def dashboard(request):

    user_id=request.session.get("user_id")

    if not user_id:

        return redirect("login")


    user = User.objects.get(id=user_id)


    return render(request, "dashboard.html", {"user": user})


def logout_view(request):

    request.session.flush()

    return redirect("login")
```

Why We Need Hashing

Right now in DB:

username: bhavna

password: 12345

Anyone who sees DB can see password.

That is unsecure.

Instead we store:

password: pbkdf2_sha256$600000$abc...longrandomstring

This is a hash.

We never store original password.

Step 1 - Import Hashing Functions

In **`views.py`**:

from django.contrib.auth.hashers import make_password, check_password

Step 2 - Modify Register View

Old version:

User.objects.create(username=username, password=password)

Replace with:

```
hashed_password = make_password(password)

User.objects.create(
    username=username,
    password=hashed_password
)
```

Now password saved in DB will look encrypted.

Test:
Register new user.
Check DB.
Password should be long hashed string.

Step 3 - Modify Login View

Old version:

```
if user.password == password:
```

Replace with:

```
if check_password(password, user.password):
```

Explanation:

- First argument → plain password from form
- Second argument → hashed password from DB

check_password() compares safely.

Full Updated Login Logic

```
def login_check(request):
    if request.method == "POST":
        username = request.POST.get("username")
        password = request.POST.get("password")

        try:
            user = User.objects.get(username=username)
```

```
        if check_password(password, user.password):
            request.session["user_id"] = user.id
            return redirect("dashboard")
        else:
            return render(request, "login.html", {"error": "Invalid password"})

    except User.DoesNotExist:
        return render(request, "login.html", {"error": "User not found"})

return render(request, "login.html")
```

Hashing ≠ Encryption

Hashing:

- One way
- Cannot decode back

Encryption:

- Two way
- Can decrypt

Passwords must be hashed, not encrypted.

Updating confirm password in register

Step 1 - Update register.html

Add confirm password field.

```
<h2>Register</h2>
```

```
<form method="post">

  {% csrf_token %}



  <input type="text" name="username" placeholder="Username"><br><br>

  <input type="password" name="password"
placeholder="Password"><br><br>

  <input type="password" name="confirm_password" placeholder="Confirm
Password"><br><br>

  <button type="submit">Register</button>

</form>



{% if error %}

<p style="color:red;">{{ error }}</p>

{% endif %}
```

Important:
`name="confirm_password"` must match view.

Step 2 - Update Register View

Modify your register view like this:

```
from django.contrib.auth.hashers import make_password
```

```
from django.shortcuts import render, redirect

from .models import User



def add_user(request):

  if request.method == "POST":

      username = request.POST.get("username")

      password = request.POST.get("password")

      confirm_password = request.POST.get("confirm_password")

      if password != confirm_password:

          return render(request, "register.html", {

              "error": "Passwords do not match"

          })

      if User.objects.filter(username=username).exists():

          return render(request, "register.html", {

              "error": "Username already exists"

          })

      hashed_password = make_password(password)

      User.objects.create(
```

```
        username=username,

        password=hashed_password

    )

    return redirect("login")

  return render(request, "register.html")
```

Inbuilt Authentication:

We will:

- Stop using your **`User`** model
- Use Django's built-in **`User`**
- Remove manual session handling
- Remove manual hashing
- Remove manual login logic

Step 1 - Import Django User Model

In views.py:

```
from django.contrib.auth.models import User
```

Now we use Django's User instead of your custom model.

Do NOT delete your old model immediately if migrations exist.
Just stop using it.

Step 2 - Replace Register View

Instead of manual hashing and confirmation logic, use **`UserCreationForm`**.

forms.py

```
from django.contrib.auth.forms import UserCreationForm

from django.contrib.auth.models import User


class RegisterForm(UserCreationForm):

    class Meta:

        model = User

        fields = ["username"]
```

Notice:

- Password confirmation
- Hashing
- Validation
  All handled automatically.

---

views.py

```
from django.contrib.auth import login

from .forms import RegisterForm


def register(request):

    if request.method == "POST":

        form = RegisterForm(request.POST)

        if form.is_valid():

            user = form.save()

            login(request, user)   # automatic session handling

            return redirect("dashboard")
```

```
    else:
        form = RegisterForm()

    return render(request, "register.html", {"form": form})
```

---

register.html

```
<h2>Register</h2>

<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Register</button>
</form>
```

Notice:
No manual confirm password logic.

Step 3 - Replace Login View

Use Django authenticate + login.

```
from django.contrib.auth import authenticate, login

def login_view(request):
    if request.method == "POST":
        username = request.POST.get("username")
```

```
        password = request.POST.get("password")

        user = authenticate(request, username=username, password=password)

        if user is not None:
            login(request, user)
            return redirect("dashboard")
        else:
            return render(request, "login.html", {"error": "Invalid credentials"})

    return render(request, "login.html")
```

Step 4 - Protect Dashboard

Replace manual session checking:

Old:

```
user_id = request.session.get("user_id")
```

Now:

```
from django.contrib.auth.decorators import login_required

@login_required
def dashboard(request):
    return render(request, "dashboard.html")
```

That’s it.

No manual checking.

Step 5 - Logout

```
from django.contrib.auth import logout


def logout_view(request):
    logout(request)
    return redirect("login")
```

No `session.flush()` needed.

IMPORTANT - Add Redirect Settings

In settings.py:

```
LOGIN_REDIRECT_URL = "dashboard"
LOGOUT_REDIRECT_URL = "login"
LOGIN_URL = "login"
```

Manual System vs Django Built-in

Manual:

- Stored password manually
- Used make_password
- Used check_password
- Managed session manually
- Wrote duplicate validation manually
- Wrote confirm password manually

Built-in:

- Hashing automatic
- Validation automatic
- Confirm password automatic
- Session handling automatic
- Authentication backend secure
- Protection decorators available

authenticate() checks password using secure backend.
login() sets session properly.
logout() clears session properly.

Django authentication builds secure applications.

Both are important.
But production must use built-in.

Adding Email field :

We will:

1. Add email field
2. Make it required
3. Save it correctly
4. Ensure uniqueness (recommended)

Clean and production-ready.

Step 1 - Update forms.py

Modify your **`RegisterForm`.**

```
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User
from django import forms

class RegisterForm(UserCreationForm):
  email = forms.EmailField(required=True)

  class Meta:
    model = User
    fields = ["username", "email", "password1", "password2"]
```

Important:

- `EmailField` automatically validates email format.
- We explicitly include `"email"` in fields.

Step 2 - Save Email Properly

Override `save()` to ensure email is stored.

```
class RegisterForm(UserCreationForm):
  email = forms.EmailField(required=True)

  class Meta:
    model = User
    fields = ["username", "email", "password1", "password2"]

  def save(self, commit=True):
```

```
    user = super().save(commit=False)
    user.email = self.cleaned_data["email"]
    if commit:
        user.save()
    return user
```

Why override?
Because `UserCreationForm` does not automatically save custom fields unless specified.

Step 3 - Template (No Major Change)

Your `register.html` remains simple:

```
<h2>Register</h2>


<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit">Register</button>
</form>
```

Email will automatically appear.

Step 4 - Make Email Unique (Best Practice)

By default, Django's User model does NOT enforce unique email.

To enforce uniqueness at form level:

Add clean_email method:

```
def clean_email(self):
```

```
email = self.cleaned_data["email"]
if User.objects.filter(email=email).exists():
    raise forms.ValidationError("Email already exists")
return email
```

Add this inside **`RegisterForm`.**

Now duplicate email registration will fail.

Step 5 - Test Properly

Test cases:

1. Invalid email format → should fail
2. Duplicate email → should fail
3. Valid email → should save
4. Check admin panel → email stored